

Introduction

Adders are integral to digital arithmetic units, with their design directly impacting computing systems' speed and power efficiency. A ripple-carry adder (RCA) is a simple yet widely used architecture that propagates the carry sequentially through a chain of 1-bit full adders.

Optimizing this design for modern high-performance systems involves addressing its inherent delay and power trade-offs. This report presents the design, implementation, and evaluation of an **8-bit dual-mode ripple-carry adder**, a versatile circuit capable of performing addition and accumulation tasks.

Background

The dual-mode ripple-carry adder introduced in this project operates in two modes:

1. **Addition Mode:** Performs standard binary addition of two 8-bit operands A and B, producing an 8-bit sum S.
2. **Accumulation Mode:** Uses the previously computed sum S as an operand, accumulating it with a new 8-bit input A to generate the updated sum.

The dual-mode functionality is controlled via a single-bit signal CTRL, while a synchronous reset feature initializes the adder state. A key design challenge is ensuring that the adder meets the objectives of high-speed operation and low-power operation.

The report outlines the design methodologies, critical path analysis, simulation results, and performance trade-offs observed during implementation. The results demonstrate the effectiveness of strategic transistor sizing and circuit optimization in meeting performance objectives.

Approach

DCVSL Design for 1-bit Full Adder

For our 1-bit Full Adder implementation, we used a DCVSL approach to optimize our circuit. Our original baseline utilized a mirror adder, as seen in the lecture slides, in which we were using the inversion property for our outputs C_o (carry-out) and S (sum). Initially, this implementation ran for very slow periods, as defined on our vector file. However, as we attempted to optimize our logic through gate sizing and reducing our terminating capacitance values, our design remained unable to meet the threshold we aimed for. Our DCVSL design for both the sum and carry-out bit was an alternative approach that had the potential to be faster and more power-efficient than our previous iteration. For this design, our logic did not include the inversion property in its implementation.

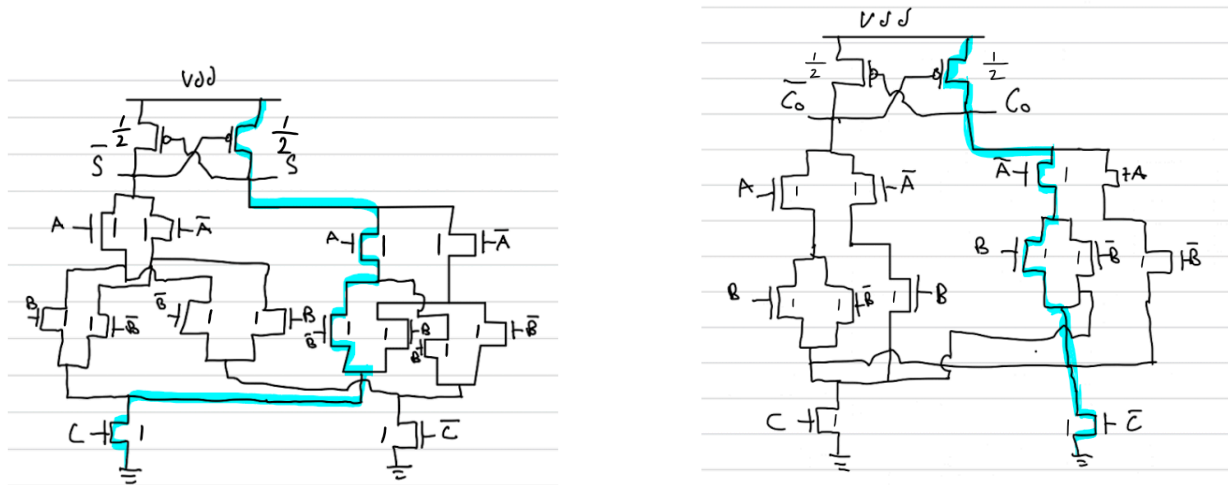


Figure 1: S and C_o logic using DCVSL

DCVSL circuits are known for their high-speed operation which can be especially advantageous in performance-critical designs like high-speed adders. In addition, Since DCVSL is a dynamic logic style that minimizes leakage current through its complementary switching paths, it can reduce static power dissipation.

Transmission Gate Design for 2:1 Multiplexer

We chose a Transmission Gate-based approach for our 2:1 Multiplexer (MUX) implementation. The TG MUX uses both NMOS and PMOS transistors in parallel, allowing the MUX to pass

both high and low logic levels without significant voltage drop. This configuration ensures the MUX can efficiently handle a wider range of input signals.

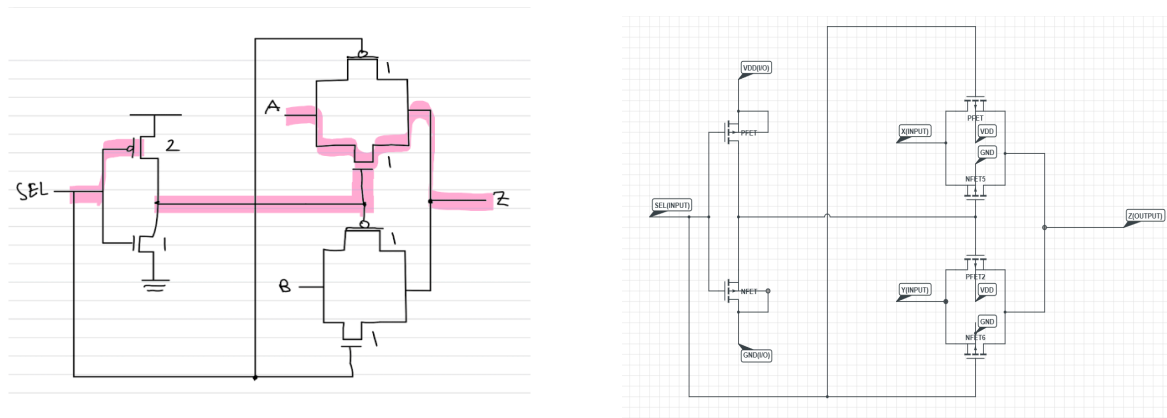


Figure 2: 2:1 MUX using Transmission Gates

The main benefit of this design is simplicity. TG-based multiplexers also offer reduced area compared to traditional logic circuits, requiring fewer components to implement the same functionality. Furthermore, these MUX circuits are power-efficient due to the low voltage drops across the transmission gates, which results in lower dynamic power consumption during operation.

Static MUX-Based Design for 1-bit Flip-Flop

For our 1-bit Flip-Flop implementation, we adopted a static MUX-based latch design due to its balance of simplicity, power efficiency, and robustness. The MUX-based latch design utilizes a 2:1 multiplexer to selectively route input signals based on the control signal (clock). This design inherently provides a clocked latch functionality, as the multiplexer routes the data input to the output when the clock is high and maintains the state when the clock is low.

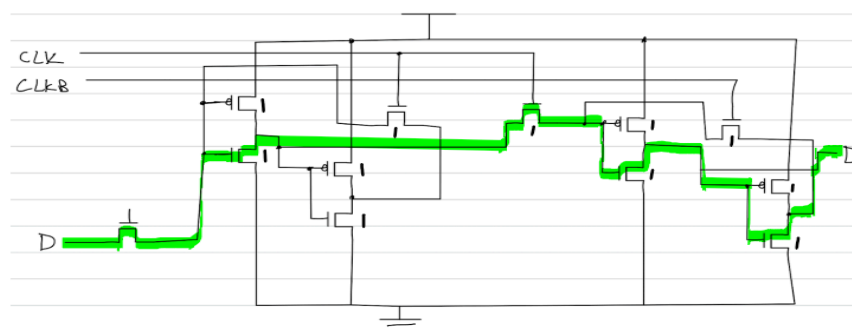


Figure 3: Mux-Based Latch

Static logic ensures that the output state is robustly retained without needing dynamic refresh cycles, addressing the drawbacks of charge leakage commonly associated with dynamic designs.

Analysis

To achieve the highest speed and lowest clock frequency for objective one, we decided to optimize the sizing of our transistors. Specifically, when looking at the design of our MUX, we noticed that SEL had to drive both transmission gates compared to just one for both inputs and therefore had a larger logical effort. As a result, we decided to size up the inverter that feeds into the transmission gates. This resulted in a minimum clock period of 1 ns or 1Gz clock frequency. Below are the results given the input patterns specified in the project description.

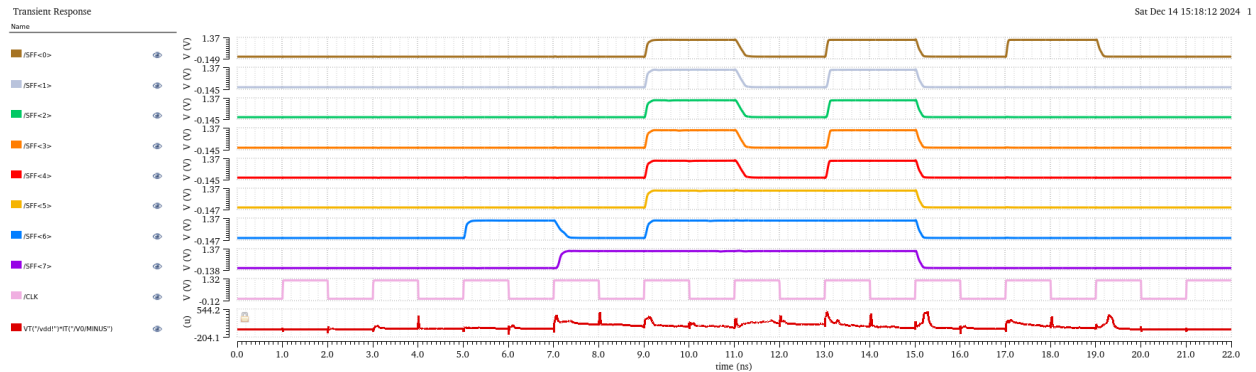
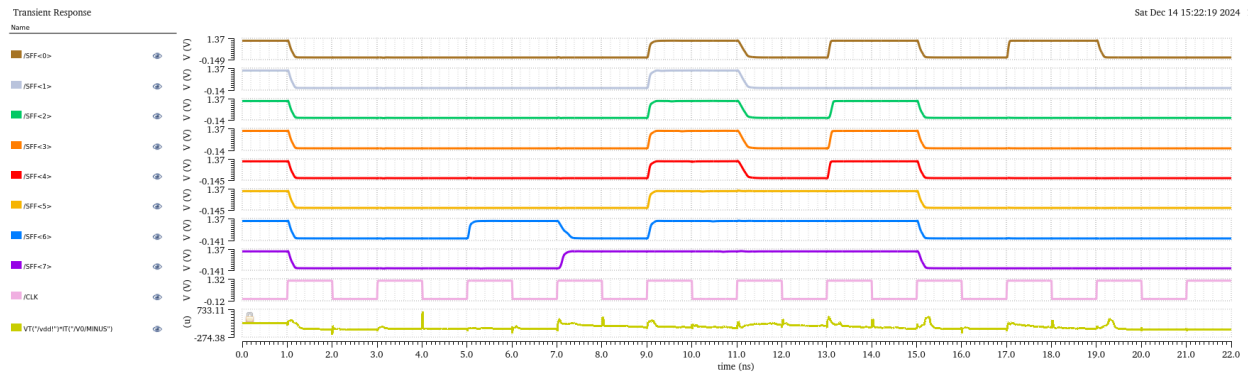


Figure 4: Objective 1 Waveforms

Our total power consumption for this design was 1.914 pJ. While sizing up our transistors led to increased performance, it was at the cost of power consumption.

For objective 2, we decided to reduce power consumption by sizing down our transistors. This led to an overall reduction in power consumption, with the total energy amounting to 1.585pJ.



Comparative Discussion

In our final design, we chose a MUX-based Flip-Flop for its simplicity and reliability over the more advanced Single-phase-clocked Flip-Flop (S²CFF). The MUX-based FF provided a straightforward and ideal implementation for our project. While the S²CFF offers advantages such as faster operation, its increased complexity outweighed the benefits for our application.

Total Width

Objective 1: 413 unit-sized transistors | 47640 nM total

Objective 2: 397 unit-sized transistors | 49560 nM total

For the full adder, we opted for DCVSL logic instead of the baseline mirror adder logic due to its superior speed and power efficiency. Although the mirror adder offered simplicity and compactness, we decided we would get our biggest increase in speed by improving full adder logic. By using DCVSL, we achieved a faster, more power-efficient design, meeting our performance goals while maintaining a manageable level of design complexity.

Conclusion

While our design successfully met the project objectives of achieving high speed and power efficiency, shortcomings remain that offer opportunities for improvement. One limitation was the implementation of the ripple-carry architecture, which inherently suffers from delay accumulation due to the sequential propagation of carry signals. A more advanced topology, such as a carry-lookahead adder, could significantly reduce this critical path delay, although it was outside the project scope. Additionally, for the MUX-based flip-flop, its simplicity came at the cost of higher power consumption compared to more advanced options like the S²CFF.